

GUÍA COMPLETA EXAMEN CIPRIANO

GUÍA COMPLETA EXAMEN CIPRIANO

PRIMERA PRÁCTICA

OBJETIVO 1:

OBJETIVO 2:

OBJETIVO 3:

- "3308:3306"

PRIMERA PRÁCTICA

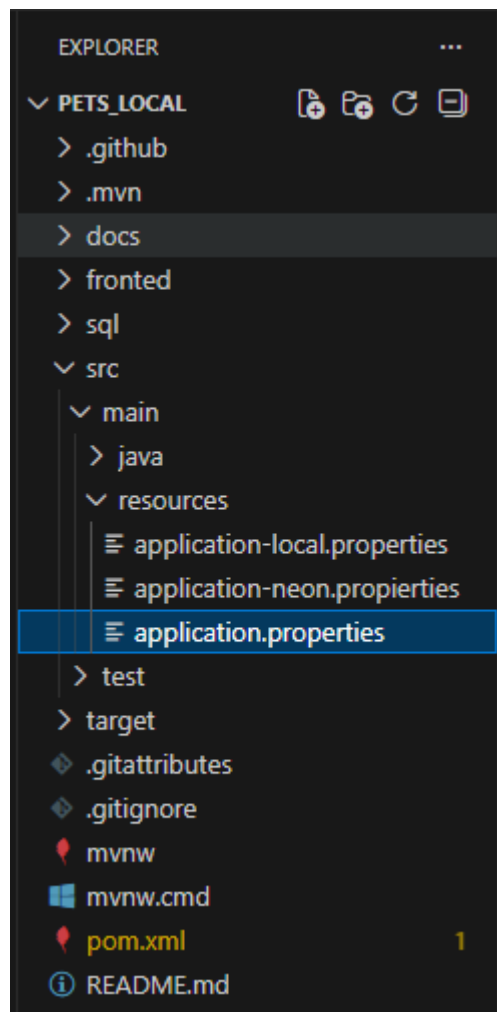
OBJETIVO 1:

Separa el contenido del fichero application.properties original en dos ficheros:

application-local.properties → entorno de desarrollo **local(MySQL)**

application-neon.properties → entorno de desarrollo **remoto(PostgreSQL/Neon)**

Estructura:



```
src > main > resources > application.properties
1  spring.application.name=despliegue-api-rest
2
3  # ESTE DE AQUI MYSQL/LOCAL
4  # spring.datasource.url=jdbc:mysql://localhost:3306/pets
5  # spring.datasource.username=root
6  # spring.datasource.password=
7  # spring.jpa.show-sql=true
8  # spring.jpa.generate-ddl=false
9  # spring.jpa.hibernate.ddl-auto=update
10 # spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
11
12 # ESTE DE AQUI ABAJO POSTGRES/NEON
13 # spring.datasource.url=jdbc:postgresql://ep-twilight-waterfall-adh2pm4g-pooler.c-2.us-east-1.aws.neon.tech/pets?sslmode=require&channelBinding=require
14 # spring.datasource.username=neondb_owner
15 # spring.datasource.password=npg_cIRIIn80vFYW
16 # spring.datasource.driver-class-name=org.postgresql.Driver
17
18 # JPA / Hibernate
19 # spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
20 # spring.jpa.hibernate.ddl-auto=validate
21 # spring.jpa.show-sql=true
22
```

En el local cambio en localhost por **host.docker.internal**

```
application-local.properties X
src > main > resources > application-local.properties
1  spring.datasource.url=jdbc:mysql://host.docker.internal:3306/pets
2  spring.datasource.username=root
3  spring.datasource.password=
4  spring.jpa.show-sql=true
5  spring.jpa.generate-ddl=false
6  spring.jpa.hibernate.ddl-auto=update
7  spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

***Si tu contraseña de sql es root, ponerla

***El puerto debe ser el que tengas [aquí](#)

OBJETIVO 2:

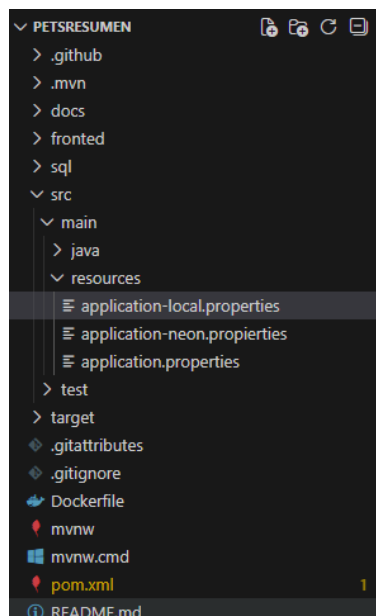
Configura un único Dockerfile válido para ambos entornos.

Este debería de darnoslo el profesor, si no, este lo crearemos en la raíz y copiaremos y pegaremos esto:

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN chmod +x mvnw || true
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Estructura:



OBJETIVO 3:

Crear los ficheros docker-compose necesarios para ejecutar la aplicación con cada perfil.

No se si cipriano nos dará los Docker-compose, debería, pero por si acaso aquí dejo el local que me funciona y el de neon:

LOCAL:

```
services:
  apiprueba:
    build: .
    container_name: apiprueba
    restart: always
    environment:
      SPRING_PROFILES_ACTIVE: local
    ports:
      - "8082:8080"

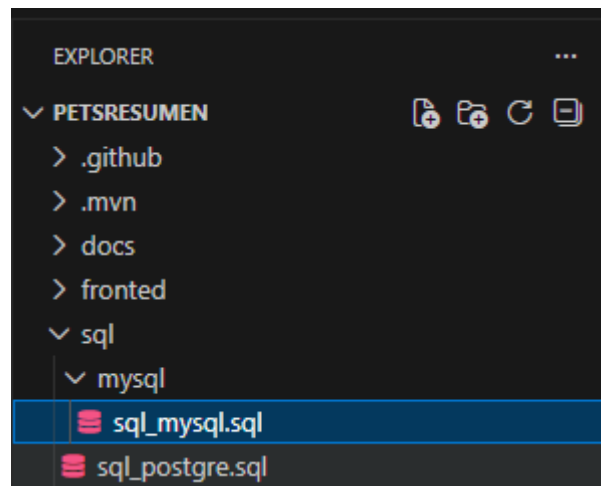
  dbprueba:
    image: mysql:8.0
    container_name: baseDatosprueba
    environment:
      MYSQL_ROOT_PASSWORD: root
    restart: always
    volumes:
      - ./sql/mysql:/docker-entrypoint-initdb.d
      - db_data:/var/lib/mysql
    ports:
      - "3308:3306"

volumes:
  db_data:
```

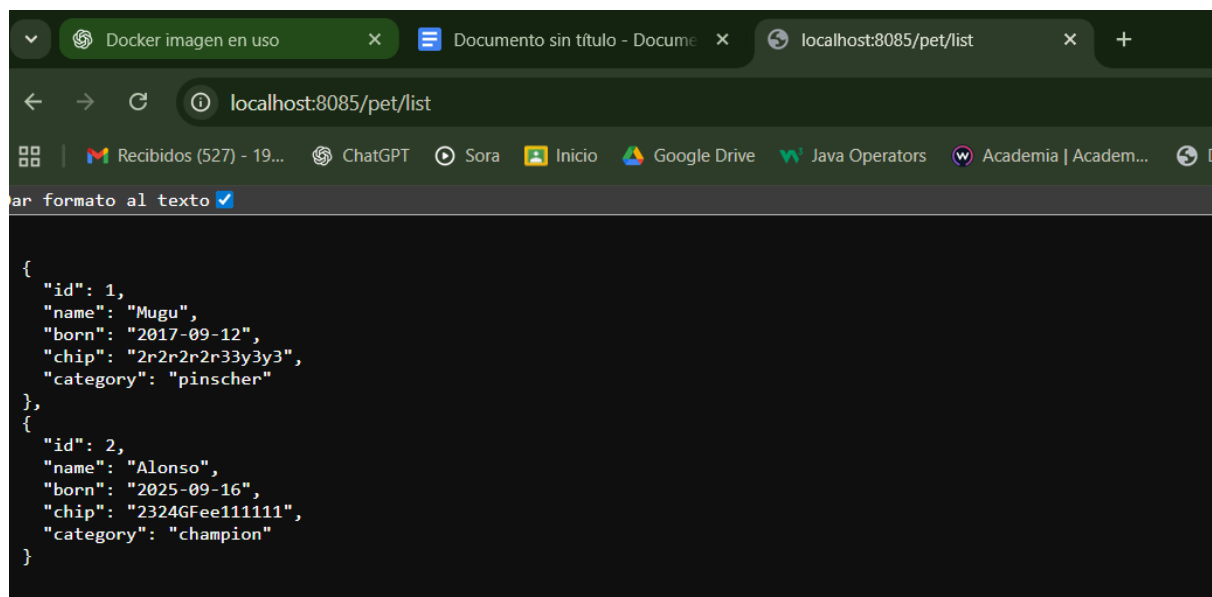
Deberíamos de poner los puertos que tengamos libres (recomendación, ir al examen con el docker.desktop vacío para no tener problemas de puertos).

Comando para construir: *docker compose -f docker-compose.local.yml up --build*

***Si nos da un error, puede ser que no tengamos en la carpeta de MySQL metido el script:



RESULTADO:

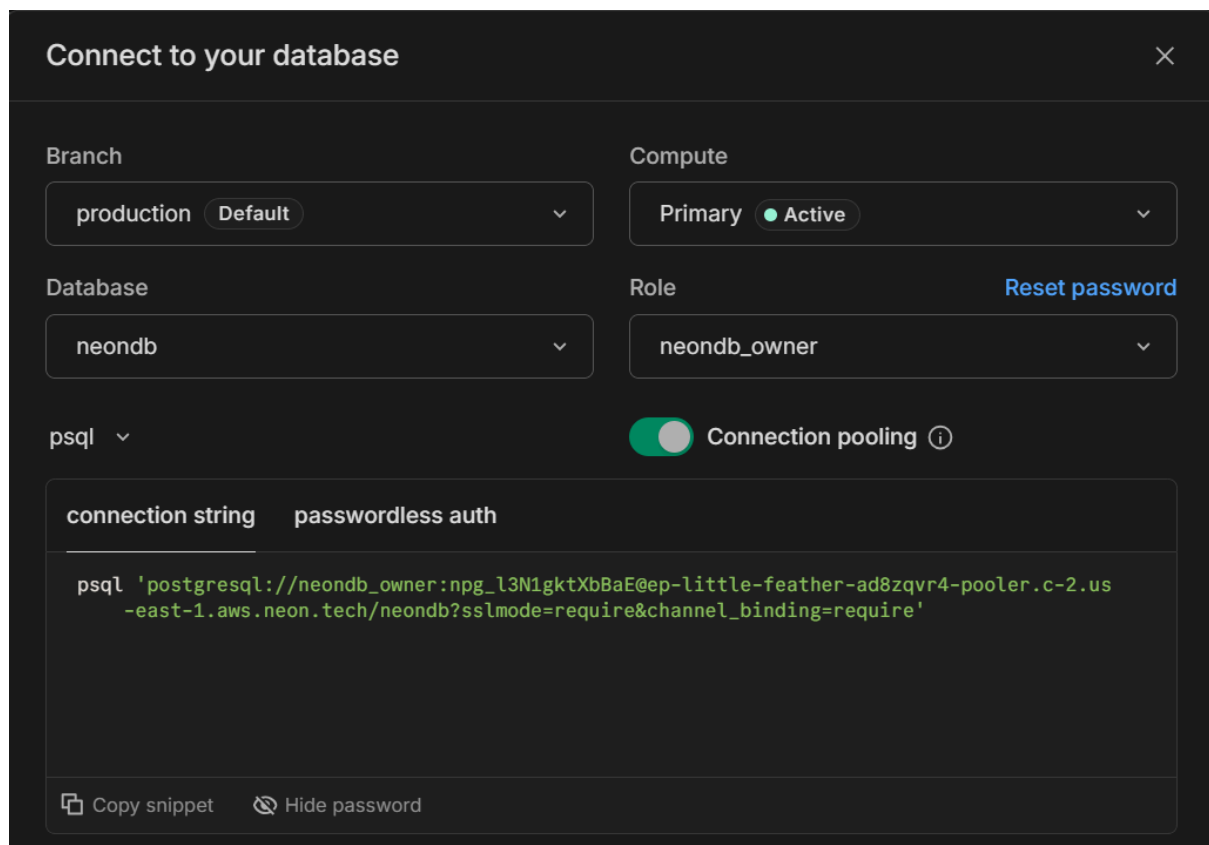


Ahora tendríamos que hacer el **neon**:

Crearemos el `docker-compose.neon.yml` y le meteremos lo siguiente:

```
services:
  api:
    build: .
    container_name: pets-api
    environment:
      SPRING_PROFILES_ACTIVE: neon
    ports:
      - "8080:8080"
```

abriremos neon y una vez creada la base de datos y metido las consultas:



Connect to your database

Branch: production (Default)

Compute: Primary (Active)

Database: neondb

Role: neondb_owner (Reset password)

psql (Connection pooling)

connection string passwordless auth

```
psql 'postgresql://neondb_owner:npg_l3N1gktXbBaE@ep-little-feather-ad8zqvr4-pooler.c-2.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require'
```

Copy snippet Hide password

Una vez tenemos todo esto lo desglosamos en el **application-neon.properties**:

host →

`postgresql://ep-little-feather-ad8zqvr4-pooler.c-2.us-east-1.aws.neon.tech/neondb?sslmode=require`

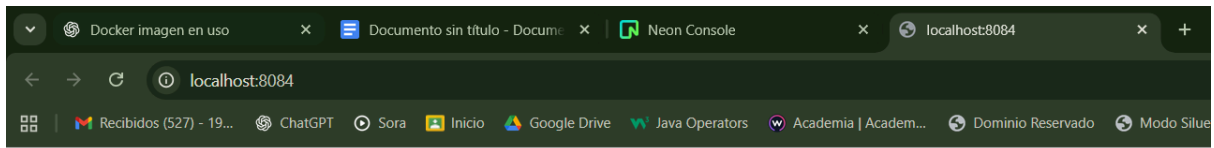
usuario → `neondb_owner`

contraseña → `npg_l3N1gktXbBaE`

Una vez puestos en su sitio

```
spring.datasource.url=jdbc:postgresql://ep-little-feather-ad8zqvr4-pooler.c-2.us-east-1.aws.neon.tech/neondb?sslmode=require
spring.datasource.username=neondb_owner
spring.datasource.password=npg_l3N1gktXbBaE
```

Comando para levantar el neon: `docker compose -f docker-compose.neon.yml up --build`



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu Dec 04 12:54:03 UTC 2025

There was an unexpected error (type=Not Found, status=404).

Si nos sale esto esta bien, ya tan solo es poner `/pet/list`

